Table of Contents

This document presents the complete Phase 0–1 architecture plan for the Jashore Blood Network. Each chapter is a discrete deliverable listed in the project brief. Chapter 1 explains how to read this document and which decisions are pre-resolved versus left open for stakeholder input.

Executive Summary & How to Read This Document . . . 5

What This Document Contains . . . 5

Pre-Resolved Decisions . . . 5

Decisions Left Open for Stakeholders . . . 6

Execution Rule . . . 6

Workspace Audit Report . . . 6

Findings . . . 6

Identified Security Risks . . . 7

Missing Requirements and Technical Decisions . . . 7

Recommended Technology Architecture . . . 8

System Components . . . 8

Why pnpm + Turborepo . . . 8

Why Prisma with Supabase . . . 9

Why Next.js on Vercel and NestJS on Render . . . 9

Monorepo Directory Structure . . . 10

Proposed Layout . . . 10

Why Three Shared Packages, Not More . . . 12

Independent Development and Deployment . . . 12

User Roles and Permission Model . . . 12

The Eight Roles . . . 12

Permission Matrix . . . 13

Role Hierarchy Diagram . . . 14

Authorization Enforcement . . . 14

Core Application Modules 15

A. Authentication and Profiles 15

B. Donor Management 15

C. Blood Donor Search 15

D. Blood Request Management 16

E. Donor Responses 16

F. Hospital and Blood Bank Directory 16

G. Notifications 17

H. Administration 17

I. Future Features (Explicitly Deferred) 17

Jashore Location Model and Seed Data 17

The Eight Upazilas of Jashore 17

Seed Data Policy 18

Coordinates and Boundary Data 18

Database Schema Design 19

Entity-Relationship Diagram 19

Tables Overview 19

Key Table Definitions (Design Level) 20

Row Level Security Strategy 22

Indexes and Query Patterns 22

Data Retention and Migration Strategy 22

API Specification 23

Endpoint Summary 23

Sample: Create Blood Request 24

Sample: Donor Search 24

Error Format (RFC 7807) 25

Rate Limits 25

CORS Configuration 26

Authentication and Authorization Plan 26

Authentication Strategy Options . . . 26
Session Handling Options . . . 26
Server-Side Token Verification . . . 27
Role-Based Authorization . . . 27
Privileged Role Changes . . . 27
Hospital Verification . . . 28

Frontend UI/UX Plan . . . 28

Public Pages . . . 29
Authenticated Pages . . . 30
Administrative Pages . . . 30
Homepage CTA Priority . . . 30
State Design . . . 31
Performance Budget . . . 31
Internationalization Readiness . . . 31

Security and Privacy Assessment . . . 31

Threat Model . . . 31
Recommended Controls . . . 32
Data Protection Plan . . . 33
Bangladesh Legal and Regulatory Review . . . 34

Testing Strategy . . . 34

Test Layers . . . 34
Accessibility Testing . . . 35
Slow-Network Testing . . . 35
Test Data . . . 35

Deployment Architecture and Free-Tier Cost Analysis . . . 36

Frontend: Vercel . . . 36
Backend: Render . . . 37
Database and Auth: Supabase . . . 37
GitHub and CI . . . 37
Free-Tier Limits and Upgrade Triggers . . . 37

Environment Variable Inventory . . . 38

Development Roadmap with Acceptance Criteria . . . 38

Phase Summaries . . . 39

Phase 0 and Phase 1 (Current Engagement) . . . 39

Phase 2: Repo Foundation (Recommended Next Task) . . . 40

Dependency-Aware Sequencing . . . 40

Open Questions and Unresolved Decisions . . . 40

Risk Register with Mitigations . . . 42

Risk Review Cadence . . . 43

Proposed Foundation Files and Next Task . . . 43

Phase 2 File Manifest . . . 43

Phase 2 Acceptance Criteria . . . 44

Recommended Next Task for Independent Approval . . . 45

After Phase 2 . . . 45

Document Status . . . 45

CHAPTER 1

Executive Summary & How to Read This Document

This document is the complete Phase 0 and Phase 1 deliverable for the **Jashore Blood Network** — a district-wide blood donor discovery and request coordination platform scoped for Jashore District, Bangladesh. It has been prepared in response to a brief that explicitly restricts this engagement to architecture, planning, workspace inspection, and foundation preparation. **No application code is written here. No database migrations are executed. No external resources are provisioned. No files are deleted or overwritten without approval.** The purpose of this document is to give stakeholders a single, reviewable artifact that captures every architectural decision, every unresolved question, and every identified risk — so that explicit approval can be granted before any implementation begins.

The Jashore Blood Network is not a blood bank and not a replacement for a licensed medical professional, blood testing, or clinical compatibility assessment. It is a coordination service: it helps potential donors make themselves discoverable, helps patients and their representatives submit verifiable blood requests, and helps volunteer coordinators and administrators keep the platform trustworthy. The architecture therefore optimizes for three things in tension with each other: *discoverability* (the right donor must be findable quickly), *privacy* (no donor's personal contact details may leak to the public), and *auditability* (every privileged action must be reconstructable from logs).

What This Document Contains

The remaining seventeen chapters correspond to the sixteen deliverables listed in the project brief (a few chapters cover two deliverables each). In order: (A) a workspace audit of the current repository, (B) the recommended technology architecture, (C) the proposed monorepo directory structure, (D and E) the database entity-relationship design and table definitions, (F) the API endpoint specification, (G) the authentication and authorization plan plus the roles and permission model, (H) the security and privacy assessment, (I) the cross-browser and accessibility testing strategy, (J) the Vercel/Render/Supabase deployment plan with free-tier cost analysis, (K) the development roadmap with acceptance criteria, (L) the open questions and unresolved decisions, (M) the risk register with mitigations, and finally (N and O) the exact list of foundation files to create during Phase 2 plus a recommended next task that can be approved independently.

Pre-Resolved Decisions

The following technical decisions were pre-resolved during the clarification round and are treated as fixed inputs for the rest of this document. They are listed here so reviewers can challenge them up front rather than discovering them buried in a later chapter.

- **Monorepo tooling — pnpm workspaces + Turborepo.** Chosen over npm/yarn workspaces + Nx for lower runtime overhead, strict dependency isolation via pnpm's node_modules layout, and Turborepo's simpler task orchestration model.
- **ORM — Prisma with Supabase connection pooler.** Chosen over raw @supabase/supabase-js for typed schema, migration history, and IDE support. The known interaction with Row Level Security (RLS) is

documented in Chapter 8: the service-role key bypasses RLS and must never reach the browser.

- **RLS strategy — defense in depth.** RLS is enabled on every table containing personally identifying information, but it is treated as a backstop, not the primary enforcement layer. NestJS guards are the primary authorization layer; RLS catches bypasses and protects against direct database access with the anon key.
- **Document format — single PDF, English only, ~60–90 pages.** Bangla localization is a future platform feature, not a documentation feature; no Bangla appears in this document.

Decisions Left Open for Stakeholders

The authentication strategy was deliberately **not** pre-resolved. Chapter 10 presents a recommendation framework rather than a single mandated choice, because the decision depends on information not available in this engagement: which SMS provider is reachable from Bangladesh within budget, whether email-first or phone-first onboarding matches the pilot audience, and whether JWT-in-httpOnly-cookie or Bearer-token session handling aligns better with the chosen deployment topology. Open question L1 in Chapter 16 captures this with an explicit decision deadline. All other technical choices in this document are recommendations that stakeholders may overrule.

Execution Rule

> **STOP — do not implement**
>
> This document ends with an explicit approval gate. After Chapter 18, no further work is performed until stakeholders approve the plan and authorize Phase 2 (Repo Foundation). The recommended next task in Chapter 18 is deliberately minimal — monorepo skeleton, tooling, empty placeholder apps — so it can be approved even if some open questions in Chapter 16 remain unresolved.

DELIVERABLE A · CHAPTER 2

Workspace Audit Report

Before proposing any structure, the existing workspace was inspected in read-only mode. The audit covered the project root, the Git repository state, the presence of any existing manifests, the existence of sensitive files, and the existence of any pre-existing source code. No files were modified, no commits were made, and no secrets were read or printed.

Findings

The workspace at /home/z/my-project/ is a **greenfield project**. There is no existing application code to adapt to, no legacy schema to migrate, and no framework lock-in to honour. The architecture plan in subsequent chapters therefore proposes a clean monorepo from scratch rather than retrofitting structure onto an existing codebase.

Path	Type	Purpose / Risk
.git/	Git repository	Initialized on main branch. No commits yet. No remote configured.
.gitignore	Config file	22 bytes. Content not modified; assumed to be a minimal default.
.env	Environment file (50 bytes)	**Risk:** Not committed but exists in working tree. Not read during this audit. **Must** be added to .gitignore before first commit and replaced with a sanitized .env.example.
download/README.md	Documentation	Tooling-managed directory for final deliverables. Not part of the project.
skills/	System tooling	Z.ai skill packages (PDF, docx, charts, etc.). **Not** part of the Jashore Blood Network project — must not be committed to the application repository.
upload/	User uploads	Contains the Bangla donor app screenshot used as UI/UX reference in Chapter 11. Should be excluded from the application repository; the screenshot itself is referenced in this document only.

Table 2.1 — Workspace inspection findings. No source code, no manifests, no framework lock-in present.

Identified Security Risks

Three concrete security risks surfaced during the audit. None is exploitable today because no application exists, but all three must be addressed before the first application commit lands on the main branch.

1. **Credential leakage via .env commit.** A 50-byte .env file already exists in the working tree. If the project is initialized without first verifying .gitignore coverage, the next git add . will commit it. Mitigation: Phase 2 begins by writing .gitignore with explicit .env, .env.*, !.env.example rules *before* any other file is created.
2. **Tooling leakage into the application repository.** The skills/ and upload/ directories contain Z.ai-internal tooling and user-supplied reference images that have no place in a public-facing application repository. Mitigation: the application repository will be rooted at a sub-directory (jashore-blood-network/) rather than at /home/z/my-project/ directly, or alternatively the project will be initialized as a fresh Git clone in a separate directory. The exact approach is an open question (Q-L2).
3. **No branch protection or signed commits.** The Git repository has no remote configured, no main branch protection, and no commit signing. Mitigation: when the GitHub repository is created in Phase 2, main must be protected to require pull-request review, and CI must pass before merge.

Missing Requirements and Technical Decisions

The audit confirms that no prior technical decisions constrain the architecture. Every choice in Chapters 3 through 14 is a fresh recommendation. The only hard external constraints are the technology stack fixed by the brief (Next.js, NestJS, Supabase, Prisma) and the hosting plan (Vercel, Render, Supabase, GitHub). All other tooling choices — test runner, linter, i18n library, observability provider — are open and will be settled during Phase 2.

DELIVERABLE B · CHAPTER 3

Recommended Technology Architecture

The technology stack is fixed by the project brief and confirmed during clarification. This chapter explains how the pieces fit together, why each pre-resolved decision was made, and where the known trade-offs lie. The architecture is deliberately conventional: there is no exotic infrastructure, no experimental framework, and no pattern that a mid-level engineer cannot maintain. The novelty of this project is in the operational and trust model, not in the technology.

System Components

The system has five runtime components and one development-time component. At runtime: a Next.js frontend hosted on Vercel, a NestJS REST API hosted on Render, a Supabase PostgreSQL database with Row Level Security, a Supabase Auth service issuing JWTs, and an external email service (Resend or SMTP relay) for transactional notifications. At development time: a shared Prisma schema and migration history that is the single source of truth for the database structure. The connection pooler (Supavisor) sits between the NestJS API and PostgreSQL to absorb connection churn from Render's free-tier sleep/wake cycles.

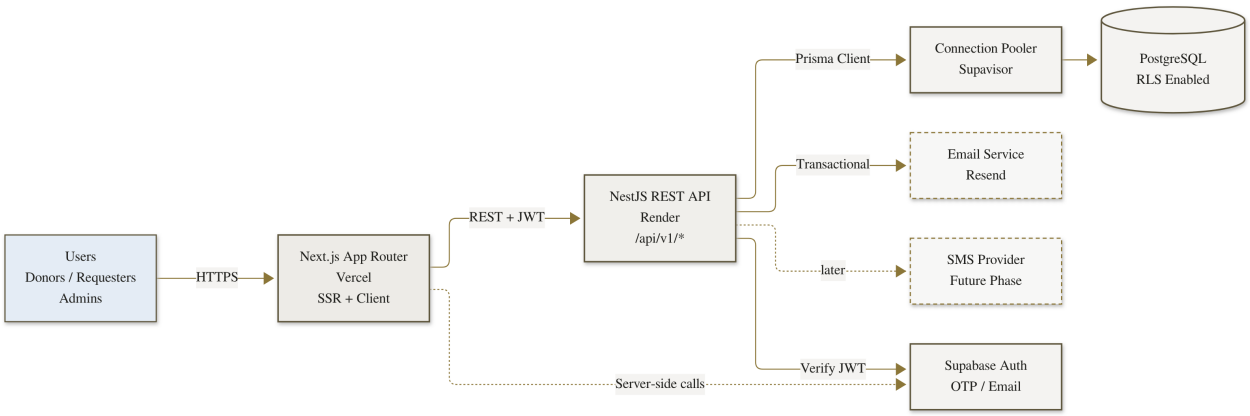


Figure 3.1 — System architecture and data flow. The Next.js frontend talks to the NestJS API over HTTPS with a JWT in an httpOnly cookie; the API verifies the JWT against Supabase Auth and reads/writes PostgreSQL via Prisma through the Supabase connection pooler. Email is transactional; SMS is deferred to a later phase.

Why pnpm + Turborepo

The monorepo uses pnpm workspaces for package resolution and Turborepo for task orchestration. pnpm was chosen over npm and yarn workspaces for three reasons. First, its symlinked node_modules layout prevents the phantom-dependency problem that has bitten every NestJS monorepo at least once: a package that imports a dependency it does not declare will fail loudly under pnpm and silently work under npm. Second, pnpm's strict peer-dependency resolution catches version mismatches early. Third, the on-disk cache makes local development noticeably faster on low-end machines — relevant because this project may be picked up by volunteer engineers working from home.

Turborepo was chosen over Nx because the project does not need generators, plugins, or a custom task graph. Turborepo gives us content-hash caching of build, test, lint, and typecheck tasks across the three workspace packages, with near-zero configuration. If the project grows to require code generation or non-trivial pipeline orchestration, Turborepo can be replaced by Nx without changing the package layout.

Why Prisma with Supabase

Prisma was chosen as the primary ORM over the raw @supabase/supabase-js client for four reasons. First, the Prisma schema file (schema.prisma) is a single, reviewable source of truth for the database structure, which makes code review and security audit substantially easier. Second, Prisma generates fully typed query builders, which eliminates the entire class of "wrong shape returned from the database" runtime errors. Third, Prisma's migration engine produces human-readable SQL files that can be inspected before being applied — critical for a project handling health data where every schema change must be auditable. Fourth, Prisma's transaction API (prisma.$transaction) makes it easy to enforce multi-table invariants such as "a blood request and its first audit-log entry must be created atomically".

The known trade-off is the interaction with Row Level Security. Prisma connects to PostgreSQL with a single connection string. If that string uses the Supabase service-role key, all RLS policies are bypassed. The mitigation is documented in Chapter 8: the NestJS API uses the service-role connection for write operations where RLS is enforced by the application layer through NestJS guards, and a separate read-only connection using the anon key plus a per-request SET LOCAL request.jwt.claims is used for user-scoped reads where RLS is the primary enforcement. **The service-role key is never shipped to the browser.**

Why Next.js on Vercel and NestJS on Render

Next.js on Vercel is the path of least resistance for a mobile-first public-facing React application: App Router for file-based routing, React Server Components for low-JavaScript page loads (critical on the 2G/3G networks common in rural Bangladesh), and Vercel's edge network for fast TTFB. The free tier is generous enough for the pilot.

NestJS on Render is the matching backend choice. NestJS's module system maps cleanly onto the domain modules in Chapter 6 (auth, donors, blood-requests, etc.), its guard and interceptor system is the natural enforcement point for the role-based authorization in Chapter 5, and its dependency injection makes the codebase testable. Render's free tier sleeps after 15 minutes of inactivity, which is acceptable for a pilot but must be flagged as a known operational risk in Chapter 14.

> **Trade-off accepted**
>
> Render's free-tier sleep behaviour means the first request after a 15-minute idle period takes ~30 seconds to respond. For an emergency blood request platform this is unacceptable in production. The mitigation is a paid Render plan (~$7/month) once the pilot exceeds ~50 active users. See Chapter 14 for the full cost-trigger table.

DELIVERABLE C · CHAPTER 4

Monorepo Directory Structure

The proposed structure is a pnpm workspace with three runnable applications (apps/web, apps/api, and a future apps/admin reserved for a separate administrative interface if the role matrix in Chapter 5 warrants it), three shared packages, and a top-level database/ directory that owns migrations and seeds. The structure is a proposal that adapts the brief's suggested layout to the pnpm+Turborepo tooling decided in Chapter 3.

Proposed Layout

```
jashore-blood-network/
|
|-- apps/
|   |-- web/                      # Next.js (App Router, mobile-first)
|   |   |-- src/
|   |   |   |-- app/              # Routes (page.tsx, layout.tsx)
|   |   |   |-- components/       # Reusable UI primitives
|   |   |   |-- features/         # Domain modules (donors, requests)
|   |   |   |-- hooks/            # Client-side data hooks
|   |   |   |-- lib/              # Utilities, constants
|   |   |   |-- services/         # API client wrappers
|   |   |   |-- types/            # Frontend-only types
|   |   |   `-- styles/           # Tailwind config + globals
|   |   |-- public/
|   |   |-- tests/                # Playwright E2E + component tests
|   |   `-- package.json
|   `-- api/                      # NestJS REST
|       |-- src/
|       |   |-- auth/             # Login, OTP, session
|       |   |-- users/            # Profile CRUD
|       |   |-- donors/           # Donor profile + search
|       |   |-- blood-requests/   # Request lifecycle
|       |   |-- request-responses/# Donor response workflow
|       |   |-- hospitals/        # Facility directory
|       |   |-- blood-banks/      # Blood bank directory
|       |   |-- locations/        # Districts/upazilas/localities
|       |   |-- notifications/    # In-app + email queue
|       |   |-- moderation/       # Abuse reports, content review
|       |   |-- admin/            # Admin-only endpoints
|       |   |-- audit/            # Audit log read + export
|       |   |-- common/           # Guards, filters, pipes, decorators
|       |   `-- config/           # Env validation, feature flags
|       |-- test/                 # Jest unit + integration
|       `-- package.json
|
|-- packages/
|   |-- shared-types/             # DTOs, enums, brand types
|   |-- validation/               # Zod schemas (shared by web + api)
|   `-- api-client/               # Typed fetch wrapper for the web app
|
|-- database/
|   |-- migrations/               # Prisma migration SQL files
|   |-- seeds/                    # Reference data only (no fake donors)
|   `-- documentation/            # ER diagrams, RLS policy docs
|
|-- docs/                         # The 12 docs from the brief
|   |-- architecture/
|   |-- database/
|   |-- api/
|   |-- security/
|   |-- deployment/
|   `-- development-roadmap/
|
|-- scripts/                      # Operational scripts (backup, etc.)
|
|-- .github/
|   `-- workflows/                # CI: lint, test, build, security scan
|
|-- .env.example                  # Template, never real credentials
|-- .gitignore
|-- .editorconfig
|-- README.md
|-- CONTRIBUTING.md
|-- package.json                  # Root: devDependencies + scripts
|-- pnpm-workspace.yaml           # Workspace declaration
`-- turbo.json                    # Task pipeline definition
```

Why Three Shared Packages, Not More

The brief lists three shared packages — shared-types, validation, and api-client — and this plan keeps exactly those three. The temptation in a fresh monorepo is to over-decompose: a constants package, a utils package, an errors package, a logger package. Each new package adds import friction, version coordination overhead, and a publish-or-not decision. The proposed three packages cover the real shared needs: shared-types holds the TypeScript types and enums that mirror the Prisma schema; validation holds Zod schemas that the API uses for input validation and the web app uses for client-side form validation; api-client is a thin typed wrapper around fetch that the web app imports. Anything else stays inside the app that uses it until a clear cross-app need emerges.

Independent Development and Deployment

Each app deploys independently. The web app builds on Vercel from the apps/web directory; Vercel's root directory setting points there. The API builds on Render from apps/api; Render's root directory setting points there. The shared packages are consumed via pnpm workspace links at build time and never published to a registry during the pilot. This means a frontend deploy does not require a backend deploy and vice versa — critical for the fast-iteration pilot phase. The database migrations are run from the developer's machine against the Supabase project (or by a CI job with the migration role key), not by either deploy pipeline, to avoid schema-drift between environments.

DELIVERABLE G (PARTIAL) · CHAPTER 5

User Roles and Permission Model

The platform defines eight user roles, designed so that a single natural person can hold multiple roles without duplicate accounts. A regular donor who today steps up to help verify incoming blood requests becomes a Volunteer Coordinator in addition to remaining a Blood Donor; the underlying profile record is unchanged, and only the user_roles join table gains a new row. This design choice eliminates the "I have two accounts" problem that plagues ad-hoc community platforms.

The Eight Roles

#	Role	Scope	Appointment
1	Public Visitor	Browse-only, no account	Self-service (anyone with a browser)
2	Registered User	Own profile, own requests	Self-service (phone/email verification)
3	Blood Donor	Own donor profile, responses to requests	Self-service (Registered User opts in to donor profile)

#	Role	Scope	Appointment
4	Volunteer Coordinator	Verify incoming requests, no PII edit	Appointed by Moderator or above
5	Hospital / Blood Bank Representative	Manage own facility profile, submit verified requests	Appointed by District Administrator after facility verification
6	Moderator	Review abuse reports, hide content, suspend users	Appointed by District Administrator
7	District Administrator	Jashore scope: verify hospitals, manage roles, view audit logs	Appointed by Super Administrator
8	Super Administrator	Platform-wide: all districts, audit logs, role management	Appointed by existing Super Administrator

Table 5.1 — The eight user roles. Roles 4–8 require authorized appointment; users cannot self-assign privileged roles.

Permission Matrix

The matrix below shows which actions each role may perform on the major resource types. The matrix is enforced by NestJS guards at the API layer and by Supabase Row Level Security at the database layer (see Chapter 8). "Own" means a user may perform the action only on records they own; "Any" means any record of that type. "Audit" means the action is logged to the audit table regardless of role.

Resource	Public	Reg. User	Donor	Volunteer	Hospital	Moderator	D. Admin	Super
Browse donor directory (limited)	Read	Read	Read	Read	Read	Read	Read	Read
View donor full contact	—	—	Own	Read+	Read+	Read	Any	Any
Create blood request	—	Own	Own	Own	Own+	—	—	—
Verify blood request	—	—	—	Any	Own+	Any	Any	Any
Respond to request	—	—	Own	—	—	—	—	—
Manage hospital profile	—	—	—	—	Own+	Any	Any	Any
Verify hospital	—	—	—	—	—	—	Any	Any
Review abuse report	—	—	—	—	—	Any	Any	Any
Suspend user	—	—	—	—	—	Any	Any	Any
Assign role	—	—	—	—	—	—	Lower	Any
Read audit log	—	—	—	—	—	—	Scope	Any

Table 5.2 — Permission matrix. "+" indicates an audit-logged variant of the action. "Scope" means scoped to the administrator's district.

Role Hierarchy Diagram

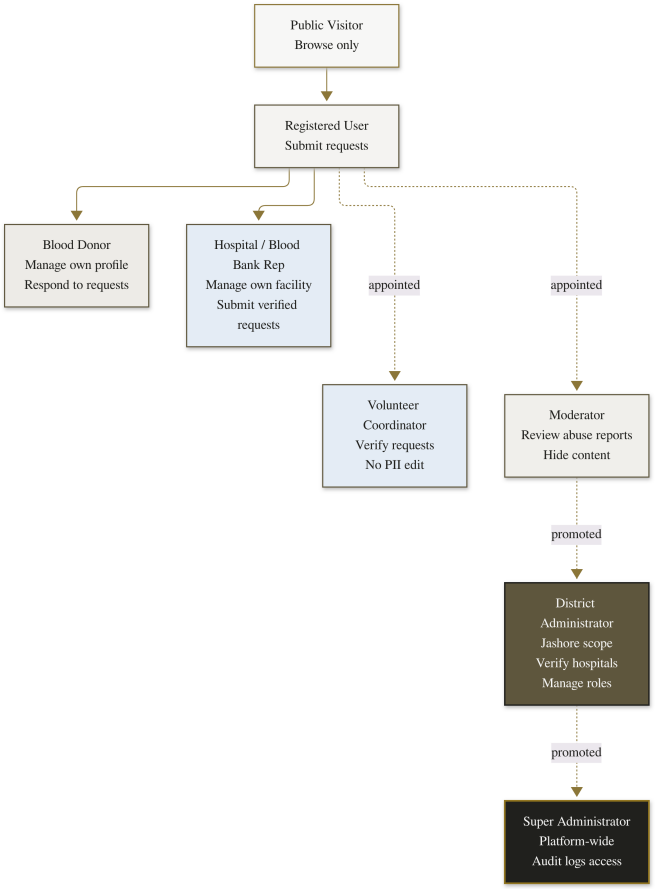


Figure 5.1 — Role progression. Solid arrows show self-service transitions; dashed arrows show appointed-role transitions. A user may hold multiple roles simultaneously (e.g. Donor + Volunteer Coordinator).

Authorization Enforcement

Three layers enforce the matrix above. **Layer 1: NestJS guards.** Every API route is annotated with a guard that checks the caller's roles against the required role set. Guards are the primary enforcement mechanism because they have access to the full request context (route params, body, authenticated user) and can make complex ownership decisions. **Layer 2: Supabase Row Level Security.** Every table containing personal data has RLS policies that mirror the guard logic. RLS is the backstop: it catches direct database access via the anon key and any guard bypass. **Layer 3: audit logging.** Every privileged action — every "Any" or "+" in the matrix — writes a row to the audit_logs table with the actor, the action, the target, the timestamp, and the request IP.

> **Iron rule — no client-side authorization**
>
> Frontend visibility is a UX concern, never a security control. Hiding an "Admin" link in the navbar does not prevent the underlying API call. Every privileged API route verifies the caller's role server-side, regardless of what the client claims. This is enforced by code review and by an integration test that exercises every privileged route with an unauthenticated and under-privileged caller.

DELIVERABLE G (PARTIAL) · CHAPTER 6

Core Application Modules

The application decomposes into nine modules, eight of which are in scope for the initial pilot and one of which (Future Features) is explicitly deferred. Each module is implemented as a NestJS module on the backend and as a feature folder on the frontend. The sections below summarize the MVP scope of each module; the full API surface is in Chapter 9 and the database tables backing each module are in Chapter 8.

A. Authentication and Profiles

Handles registration, login, logout, session management, email and phone verification, profile management, account recovery, and consent management. Phone verification uses Supabase Auth's OTP flow; we do **not** assume a phone number is verified until the OTP has been confirmed. Email verification is supported but secondary. Consent records capture explicit opt-in for donor discoverability and for notification channels (in-app, email, future SMS); a user may withdraw consent at any time, which immediately removes their donor profile from search results even though the underlying record is retained for audit purposes.

B. Donor Management

Donor registration captures blood group, district, upazila, optional locality, last donation date (voluntarily provided), contact preferences, availability status, availability confirmation timestamp, temporary unavailability windows, notification preferences, and profile visibility settings. The availability confirmation timestamp is a key field: a donor's "available" status decays after a configurable period (default 30 days) unless the donor reconfirms, which prevents stale donor records from polluting search results. Account deactivation and deletion workflows respect the data retention policy in Chapter 12: deletion is a soft delete followed by a hard delete after a 90-day grace period.

C. Blood Donor Search

Search supports blood group filtering, upazila filtering, optional locality filtering, availability filtering, and pagination. Distance-based search is an open question (Q-L3): the simplest viable approach is upazila-scoped filtering, which avoids the complexity of PostGIS and is good enough for a district-scoped pilot. Search results are ranked by availability recency and then by last donation date. **Search results never expose exact residential addresses or private contact details.** A requester who wants to contact a donor must submit a blood request first; the donor then chooses whether to respond.

> **Critical rule — no medical eligibility claims**
>
> The platform must never describe a donor as "medically eligible" based only on profile data. The donor profile captures self-reported information that may be out of date, incomplete, or wrong. The only claim the platform makes about a donor is "this person has indicated willingness to be contacted about blood donation". Clinical eligibility is determined by the blood bank at the time of donation.

D. Blood Request Management

A blood request captures the required blood group, the number of units requested, the hospital or collection facility, the general location, the required date and time, the requester's contact information, the verification status, the expiration timestamp, and the cancellation state. Duplicate request detection runs on submission: if the same requester submits a request with the same blood group, hospital, and required date within a 24-hour window, the second submission is flagged for moderator review rather than rejected outright. Request history is retained indefinitely; the full lifecycle is shown below.

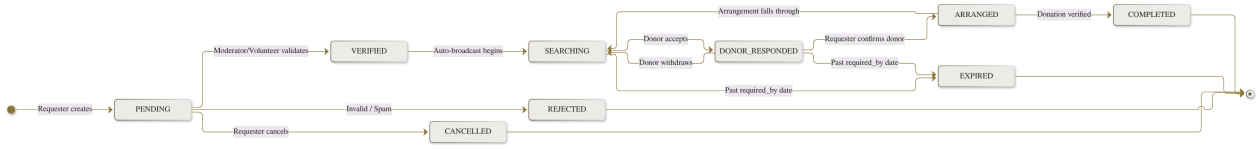


Figure 6.1 — Blood request lifecycle. The proposed state machine separates request states (left column) from donor-response states (not shown; tracked in the request_responses table). Every transition is guarded by role and audit-logged.

The lifecycle deserves a note on the SEARCHING ↔ DONOR_RESPONDED transition. The brief asks whether request and donor-response states should be separated; the recommendation is **yes**: the request itself moves through PENDING → VERIFIED → SEARCHING → ARRANGED → COMPLETED, while each individual donor response moves through its own PENDING → ACCEPTED → WITHDRAWN/CONFIRMED lifecycle in the request_responses table. This avoids the ambiguity of "the request is in DONOR_RESPONDED state — but which donor responded, and is the requester still searching?"

E. Donor Responses

A donor response records that a specific donor has accepted or declined a specific request. The response carries its own status (PENDING, ACCEPTED, DECLINED, WITHDRAWN, CONFIRMED). Duplicate responses are prevented by a unique constraint on (request_id, donor_id). A donor may withdraw an accepted response up until the requester confirms them as the arranged donor. After confirmation, the response is locked and the request transitions to ARRANGED. Completion feedback (whether the donation actually happened) is collected after the required date and feeds back into the donor's availability timestamp.

F. Hospital and Blood Bank Directory

Hospitals and blood banks are listed in a directory with facility name, address, contact details, services offered, operating hours, verification status, the authorized facility representative, and an update history. Verification status is a three-state enum (UNVERIFIED, VERIFIED, REJECTED) and can only be set by a District Administrator. **Actual blood inventory management is out of scope for the MVP** unless qualified stakeholders explicitly approve its requirements; tracking real-time blood bag inventory requires integration with hospital information systems that the pilot cannot assume.

G. Notifications

In-app notifications are the MVP channel: a notifications table records events (new request matching a donor's blood group, response received, request verified, etc.) and the web app polls or subscribes to them. Email notifications are configured where the user has verified an email address and opted in. SMS integration is explicitly deferred to a later phase — the brief correctly notes that free SMS service is not available, and SMS providers reachable from Bangladesh (e.g. SSL Wireless, Grameenphone) require commercial contracts. The notification system has retry and failure handling, rate limiting (no user receives more than N notifications per hour), and delivery logs that exclude sensitive payload data.

H. Administration

The administration module exposes a dashboard, user and donor management, request moderation, hospital verification, abuse report review, duplicate request review, audit log viewing, operational reports, and role management. Every administrative action is audit-logged. The dashboard shows operational metrics (active requests, available donors by blood group, pending verifications, open abuse reports) but does **not** show fake counters or vanity metrics.

I. Future Features (Explicitly Deferred)

Progressive Web App support, Bangla interface, Android application, volunteer coordination workflows, multi-district expansion, and approved blood bank inventory integration are all explicitly out of scope for the initial pilot. They are listed here so that the architecture does not inadvertently prevent them, but they must not be implemented during Phases 2 through 10. The architecture in this document is designed so that adding any of these features is a clean extension rather than a refactor.

CHAPTER 7

Jashore Location Model and Seed Data

The location model is normalized into four levels: country, district, upazila, and locality. For the pilot, only Bangladesh, Jashore District, and the eight upazilas of Jashore are seeded. Localities are an optional fourth level that donors may specify (e.g. a ward or a well-known neighbourhood within an upazila) but are not pre-seeded — a locality is created when the first donor in that locality specifies it, and the moderation team reviews new locality submissions before they appear in the public locality picker.

The Eight Upazilas of Jashore

#	Upazila (English)	Notes
1	Jashore Sadar	District headquarters; contains Jashore town.
2	Abhaynagar	South of Jashore Sadar.
3	Bagherpara	East of Jashore Sadar.

#	Upazila (English)	Notes
4	Chaugachha	West of Jashore Sadar; borders India.
5	Jhikargachha	Northwest of Jashore Sadar.
6	Keshabpur	South of Jashore Sadar.
7	Manirampur	East of Jashore Sadar.
8	Sharsha	Southwest of Jashore Sadar; borders India.

Table 7.1 — The eight upazilas of Jashore District to be seeded as reference data. Official administrative names must be verified against Bangladesh Local Government sources before production deployment.

Seed Data Policy

The seed data set is deliberately minimal. It contains only legitimate reference data: the country record for Bangladesh, the district record for Jashore, and the eight upazila records listed above. **The seed data does not contain any donor records, any hospital records, any blood bank records, any blood request records, or any user accounts.** A separate seed-fictional.ts script may be maintained for development and demo purposes, but its output is clearly labelled as fictional in every table that stores it (via an is_fictional boolean column) and it is never run against the production database.

> **Iron rule — no invented data**
>
> The architecture, documentation, and seed scripts must not invent village coordinates, hospital details, donor records, or facility availability. Inventing such data creates a false sense of platform completeness and risks shipping fictional entries into production. If demo data is needed for a stakeholder presentation, it lives in a separate, labelled, never-deployed seed file.

Coordinates and Boundary Data

Upazila records carry optional latitude and longitude fields for the upazila headquarters (a single point, not a polygon). Polygon boundary data is not stored: it would require integration with a boundary dataset (e.g. GeoBoundaries) that is out of scope for the pilot, and the only use case for polygons — "show me donors within this upazila's boundary" — is satisfied by the upazila filter on the donor profile. Village-level coordinates are not seeded. Hospital geocoding is captured when a facility is verified by an administrator; the geocoding source (manual entry vs. forward geocoding from address) is recorded for auditability.

DELIVERABLES D, E · CHAPTER 8

Database Schema Design

The database is a normalized PostgreSQL schema hosted on Supabase. All tables use UUID primary keys (generated by gen_random_uuid()) to prevent enumeration attacks and to allow offline ID generation. All tables have created_at and updated_at timestamp columns; tables that own user-supplied data also have deleted_at for soft deletion. Blood group is an enum type, not free text — this is critical because the search filter depends on exact equality. The schema below is a design-level overview; the column-level definitions are in Section 8.3.

Entity-Relationship Diagram

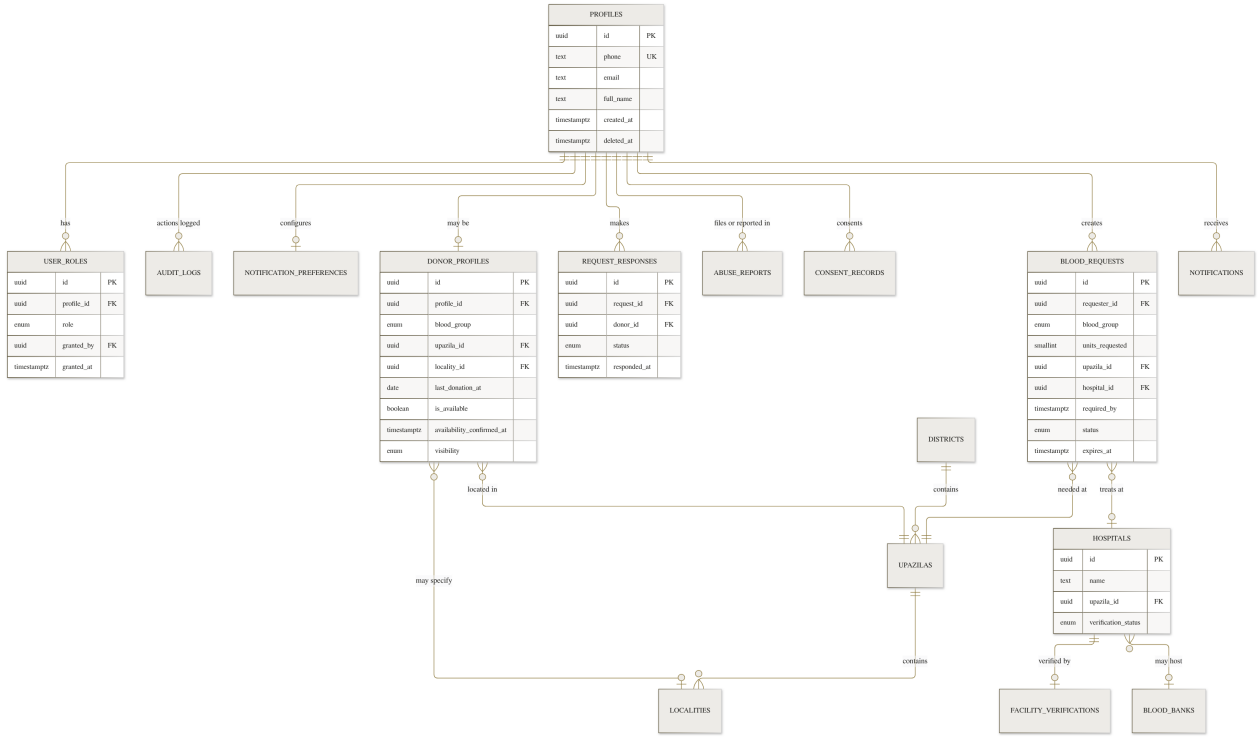


Figure 8.1 — Entity-relationship diagram (simplified; not every column is shown). Bold names are primary keys; arrows point from foreign key to primary key. The profiles table is the central entity; all user-owned data references back to it.

Tables Overview

Table	Purpose	Soft Delete	RLS
profiles	User accounts (mirrors Supabase Auth)	Yes	Yes
user_roles	Role assignments (many-to-many with profiles)	No	Yes
districts	Reference: districts of Bangladesh	No	No (public read)
upazilas	Reference: upazilas of Jashore	No	No (public read)

Table	Purpose	Soft Delete	RLS
localities	User-submitted, moderator-approved localities	Yes	Yes
donor_profiles	Donor-specific data (1:1 with profiles)	Yes	Yes
blood_requests	Blood requests submitted by users	Yes	Yes
request_responses	Donor responses to requests	Yes	Yes
hospitals	Hospital directory entries	Yes	Yes
blood_banks	Blood bank directory entries (optional, linked to hospitals)	Yes	Yes
facility_verifications	Verification records for hospitals	No	Yes
notifications	In-app notifications	Yes	Yes
notification_preferences	Per-user notification channel preferences	No	Yes
consent_records	Consent grants and withdrawals (append-only)	No	Yes
abuse_reports	User-submitted abuse reports	No	Yes
audit_logs	Append-only audit trail	No	Yes (admin read only)

Table 8.1 — All sixteen tables in the schema. RLS is enabled on every table containing user data; the two reference tables (districts, upazilas) are publicly readable.

Key Table Definitions (Design Level)

profiles

```
id              uuid PK  default gen_random_uuid()
phone           text UNIQUE                      -- E.164 format (+880...)
email           text UNIQUE NULL                 -- optional
full_name       text NOT NULL
is_active       boolean NOT NULL default true
created_at      timestamptz NOT NULL default now()
updated_at      timestamptz NOT NULL default now()
deleted_at      timestamptz NULL                 -- soft delete
-- RLS: user can read own row; admins can read all in scope
```

donor_profiles

```
id                          uuid PK
profile_id                  uuid FK -> profiles(id) UNIQUE
blood_group                 blood_group_enum NOT NULL  -- A+, A-, B+...
upazila_id                  uuid FK -> upazilas(id) NOT NULL
locality_id                 uuid FK -> localities(id) NULL
last_donation_at            date NULL                  -- voluntary
is_available                boolean NOT NULL default true
availability_confirmed_at   timestamptz NOT NULL default now()
visibility                  visibility_enum NOT NULL default 'LISTED'
contact_preference          contact_pref_enum NOT NULL default 'IN_APP'
created_at, updated_at, deleted_at  timestamptz
-- Indexes: (blood_group, upazila_id, is_available) for search
--          (availability_confirmed_at) for stale-availability sweep
```

blood_requests

```
id                uuid PK
requester_id      uuid FK -> profiles(id) NOT NULL
blood_group       blood_group_enum NOT NULL
units_requested   smallint NOT NULL CHECK (between 1 and 10)
upazila_id        uuid FK -> upazilas(id) NOT NULL
hospital_id       uuid FK -> hospitals(id) NULL
required_by       timestamptz NOT NULL  -- must be future
status            request_status_enum NOT NULL default 'PENDING'
expires_at        timestamptz NOT NULL   -- auto = required_by + 24h
verification_note text NULL
created_at, updated_at, deleted_at  timestamptz
-- Indexes: (status, blood_group, upazila_id) for donor-matching query
--          (requester_id, created_at desc) for "my requests" view
```

request_responses

```
id            uuid PK
request_id    uuid FK -> blood_requests(id) NOT NULL
donor_id      uuid FK -> profiles(id) NOT NULL
status        response_status_enum NOT NULL default 'PENDING'
responded_at  timestamptz NOT NULL default now()
updated_at    timestamptz NOT NULL default now()
withdrawn_at  timestamptz NULL
-- UNIQUE (request_id, donor_id) prevents duplicate responses
-- Index: (donor_id, status) for "my responses" view
```

audit_logs

```
id          uuid PK
actor_id    uuid FK -> profiles(id) NULL   -- NULL for system actions
action      text NOT NULL                  -- e.g. 'ROLE_ASSIGNED'
target_type text NOT NULL                  -- e.g. 'profile'
target_id   uuid NULL
ip_address  inet NULL
user_agent  text NULL
metadata    jsonb NOT NULL default '{}'
created_at  timestamptz NOT NULL default now()
-- Append-only: no UPDATE, no DELETE
-- RLS: admins read in scope; nobody writes via API (server-side only)
```

Row Level Security Strategy

RLS is enabled on every table containing personal data. The strategy is defense-in-depth: NestJS guards are the primary enforcement layer (they have full request context and can make complex ownership decisions), and RLS is the backstop that catches direct database access with the anon key. The NestJS API uses two database connections: a service-role connection for write operations and admin reads where RLS is bypassed and the application layer is the sole enforcer, and an anon-key connection for user-scoped reads where RLS is the primary enforcer and the request's JWT is passed via SET LOCAL request.jwt.claims.

> **Iron rule — service-role key never reaches the browser**
>
> The Supabase service-role key bypasses all RLS policies. It is stored only as a backend environment variable on Render, never in any NEXT_PUBLIC_ variable, never in client-side code, and never in the Vercel frontend environment. The frontend uses the anon key, which is safe to expose because RLS policies prevent it from reading other users' data.

Indexes and Query Patterns

The two hot paths are donor search (filter by blood group + upazila + availability, paginated) and request matching (find donors in the same upazila with the matching blood group who are available and have not already responded). A composite index on (blood_group, upazila_id, is_available) in donor_profiles supports both. A composite index on (status, blood_group, upazila_id) in blood_requests supports the donor-side "find open requests matching my blood group" query. The audit_logs table is indexed on (actor_id, created_at desc) for "show me this user's recent actions" and on (target_type, target_id, created_at desc) for "show me the history of this object".

Data Retention and Migration Strategy

Soft-deleted records are purged after a 90-day grace period by a daily scheduled job. Audit logs are retained indefinitely (or until a legal retention limit is established — see Chapter 12). Notification records are purged after 30 days. Migration history is owned by Prisma; each migration is a versioned SQL file in database/migrations/, reviewed in pull requests, and applied by a CI job with the migration role key before the deploy of the corresponding API version. Migrations are forward-only; rollbacks are done by writing a new migration that reverses the previous one, never by reverting a migration file.

DELIVERABLE F · CHAPTER 9

API Specification

The API is REST, versioned under /api/v1, served by NestJS on Render. All requests and responses are JSON; all timestamps are ISO 8601 with timezone offset. Authentication is via JWT in an httpOnly cookie (preferred) or an Authorization: Bearer header (for non-browser clients). CORS is configured to allow the Vercel frontend origin with credentials; **wildcard CORS with credentials is forbidden**. Pagination is cursor-based for donor search (high cardinality) and offset-based for everything else. Errors follow RFC 7807 problem-details format.

Endpoint Summary

Path	Methods	Auth	Purpose
/api/v1/auth/otp/request	POST	None	Request OTP for phone login
/api/v1/auth/otp/verify	POST	None	Verify OTP, receive session cookie
/api/v1/auth/logout	POST	Any	Invalidate session
/api/v1/auth/me	GET	Any	Get current user and roles
/api/v1/users/me	GET, PATCH	Any	Read or update own profile
/api/v1/donors	GET	Registered+	Search donors (filtered directory)
/api/v1/donors/me	GET, POST, PATCH, DELETE	Donor	Manage own donor profile
/api/v1/blood-requests	GET, POST	Registered+	List or create requests
/api/v1/blood-requests/{id}	GET, PATCH	Owner/Mod+	Read or update a request
/api/v1/blood-requests/{id}/verify	POST	Volunteer+	Verify a request
/api/v1/blood-requests/{id}/cancel	POST	Owner/Mod+	Cancel a request
/api/v1/request-responses	POST	Donor	Respond to a request
/api/v1/request-responses/{id}	GET, PATCH	Owner	Read or withdraw a response
/api/v1/hospitals	GET	Public	List hospitals (verified only)
/api/v1/hospitals/{id}	GET	Public	Read a hospital
/api/v1/hospitals/{id}	PATCH	Hospital Rep+	Update own hospital
/api/v1/locations/upazilas	GET	Public	List Jashore upazilas
/api/v1/notifications	GET	Any	List own notifications
/api/v1/notifications/{id}/read	POST	Owner	Mark as read
/api/v1/admin/users	GET	Admin	List users (paginated)

Path	Methods	Auth	Purpose
/api/v1/admin/audit-logs	GET	Admin	Query audit log
/api/v1/admin/reports/abuse	GET	Mod+	List abuse reports

Table 9.1 — API endpoint summary. "Registered+" means any authenticated user; "Owner/Mod+" means the resource owner or a moderator and above; "Admin" means District Administrator or above.

Sample: Create Blood Request

The most security-sensitive write endpoint is create-blood-request because it is reachable by any registered user and it triggers downstream notification to donors. The request body is validated by a Zod schema shared with the frontend, the required-by timestamp must be in the future, and a duplicate-detection check runs before insertion.

```
POST /api/v1/blood-requests
Cookie: session=eyJhbGc...
Content-Type: application/json

{
  "blood_group": "O+",
  "units_requested": 2,
  "upazila_id": "5f8d...e2",
  "hospital_id": "1c4a...77",
  "required_by": "2026-11-15T10:00:00+06:00",
  "contact_note": "Family member, please call before coming."
}

Response: 201 Created
{
  "id": "a3b9...",
  "status": "PENDING",
  "expires_at": "2026-11-16T10:00:00+06:00",
  "verification_status": "UNVERIFIED",
  "created_at": "2026-10-26T14:32:11+06:00"
}
```

Sample: Donor Search

```
GET /api/v1/donors?blood_group=O%2B&upazila;_id=5f8d...e2&page;=1 HTTP/1.1
Cookie: session=eyJhbGc...

Response: 200 OK
{
  "items": [
    {
      "id": "9e21...",
      "blood_group": "O+",
      "upazila": "Jashore Sadar",
      "locality": "Basundara",
      "last_donation_at": "2026-04-10",
      "is_available": true,
      "contact_action": "submit_request"
      // NOTE: no phone, no email, no full_name, no address
    }
    // ... 19 more
  ],
  "page": 1,
  "page_size": 20,
  "total_count": 47
}
```

Error Format (RFC 7807)

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json

{
  "type": "https://jashore-blood-network.org/errors/validation",
  "title": "Validation failed",
  "status": 422,
  "detail": "required_by must be in the future",
  "instance": "/api/v1/blood-requests",
  "errors": [
    {"field": "required_by", "code": "must_be_future"}
  ]
}
```

Rate Limits

Endpoint class	Limit	Rationale
OTP request	3 / hour / phone	Prevent OTP-sms cost abuse
Login attempt	10 / hour / IP	Slow credential stuffing
Donor search	60 / minute / user	Prevent donor enumeration
Create blood request	5 / day / user	Prevent spam requests
Create response	20 / hour / donor	Prevent response spam
Other read endpoints	300 / minute / user	Generous; tightens on abuse
Admin endpoints	120 / minute / admin	Sufficient for dashboard use

Table 9.2 — Per-endpoint rate limits. Limits are enforced in NestJS via a sliding-window counter stored in Redis (Render add-on) or, during the pilot, in PostgreSQL.

CORS Configuration

CORS allows the Vercel frontend origin (and a configurable list of preview-deploy origins) with credentials. Preflight is cached for one hour. The Access-Control-Allow-Headers list is explicit (Content-Type, Authorization, X-Requested-With); * is not used. The Access-Control-Allow-Origin header is reflected from the allow-list, never wildcarded. This configuration is enforced by both NestJS middleware and a deploy-time check that fails the build if wildcard CORS is detected.

DELIVERABLE G · CHAPTER 10

Authentication and Authorization Plan

Authentication is provided by Supabase Auth. The authorization model (roles, permissions, enforcement) is implemented in the NestJS API with Supabase Row Level Security as a defense-in-depth backstop. The exact authentication strategy was deliberately **not** pre-resolved in the clarification round; this chapter presents a recommendation framework and the open question Q-L1 in Chapter 16 captures the decision deadline.

Authentication Strategy Options

Option	Pros	Cons	Recommendation
Phone + OTP only	Matches Bangladesh mobile usage; no password to leak; OTP confirms possession	Depends on SMS provider reachable from BD; recovery requires backup channel	Strong fit for the pilot if SMS budget confirmed
Email + password	No SMS dependency; well-understood; Supabase supports out of the box	Password reuse risk; does not match mobile-first usage; email verification rates lower	Workable fallback if SMS unavailable
Hybrid (phone primary, email secondary)	Best of both; phone for login, email for recovery and notifications	More implementation surface; two verification flows to maintain	Recommended long-term; pilot may start phone-only

Table 10.1 — Authentication strategy options. The decision is captured as open question Q-L1 in Chapter 16.

Session Handling Options

Independent of the login mechanism is the session handling choice. The recommendation is JWT in an httpOnly, Secure, SameSite=Lax cookie. The JWT is issued by Supabase Auth after OTP verification, validated by NestJS on every request, and refreshed via a refresh-token cookie with a sliding 7-day window. The alternative — Bearer token in an Authorization header managed by the SPA — requires the JWT to live in JavaScript-accessible storage (localStorage or a closure), which exposes it to XSS exfiltration. The cookie approach is not XSS-proof but it raises the bar significantly.

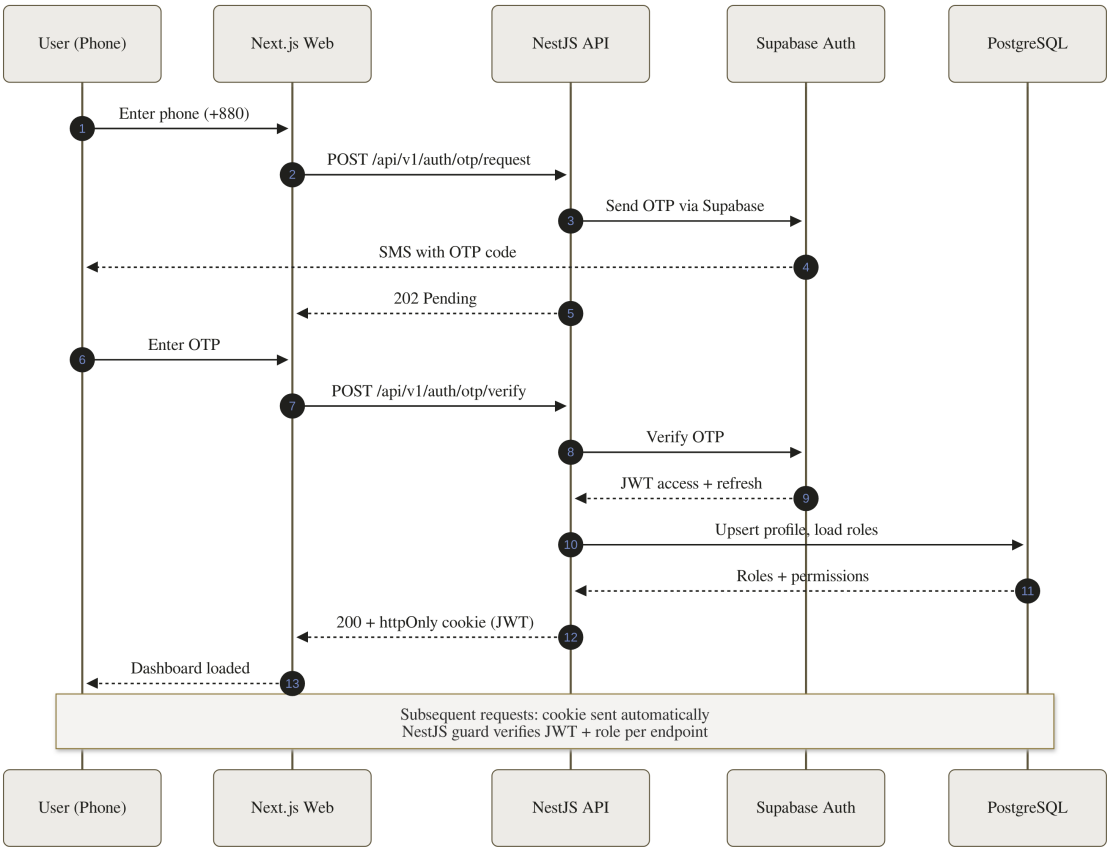


Figure 10.1 — Authentication flow for phone OTP login. The session cookie is set as httpOnly + Secure + SameSite=Lax; subsequent requests carry it automatically and NestJS guards verify the JWT and role on every privileged endpoint.

Server-Side Token Verification

Every NestJS request passes through an AuthMiddleware that extracts the JWT from the cookie (or Authorization header), verifies its signature against the Supabase JWT secret, checks its expiry, and loads the user's roles from the database. The resulting AuthenticatedUser object is attached to the request and is available to guards. Token verification happens server-side; the frontend never sees the JWT and cannot tamper with it.

Role-Based Authorization

Authorization is enforced by NestJS guards. Each guard implements CanActivate and checks the user's roles against the required role set for the route. Guards have access to the full request context, so they can make ownership decisions ("is this user the owner of this blood request?") by querying the database. Complex authorization rules (e.g. "a Hospital Representative can update their own hospital but only fields X, Y, Z") are implemented as policies consumed by guards; the policies are unit-tested in isolation.

Privileged Role Changes

A user cannot self-assign any privileged role. Role assignment is performed by an authorized administrator via the POST /api/v1/admin/users/{id}/roles endpoint, which is guarded by the AssignRole policy. The policy enforces a strict ordering: a District Administrator may assign roles up to and including Moderator; only a Super

Administrator may assign District Administrator or Super Administrator roles. Every role change writes an ROLE_ASSIGNED audit log entry with the actor, the target, the role, and the timestamp.

Hospital Verification

Hospital verification is the process by which a District Administrator confirms that a hospital directory entry corresponds to a real, licensed facility and that the user claiming to represent it is authorized to do so. Verification requires out-of-band contact (phone call to the hospital's published number, not the number the requester provided). The verification record captures who verified, when, what evidence was reviewed, and the resulting status. Only verified hospitals appear in the public directory.

CHAPTER 11

Frontend UI/UX Plan

The frontend is mobile-first, accessible, and optimized for low-end Android devices on 2G/3G networks — the realistic pilot audience in Jashore. The visual language is restrained: a single accent colour, generous whitespace, clear typography, and no animations that could be misread as medical urgency. The Bangla donor app screenshot attached to the brief (shown below) is a useful reference for the mobile-first layout and the red-on-white colour palette; the Jashore design adapts this language while removing the "emergency countdown" and "live donor counter" patterns that the brief explicitly forbids.

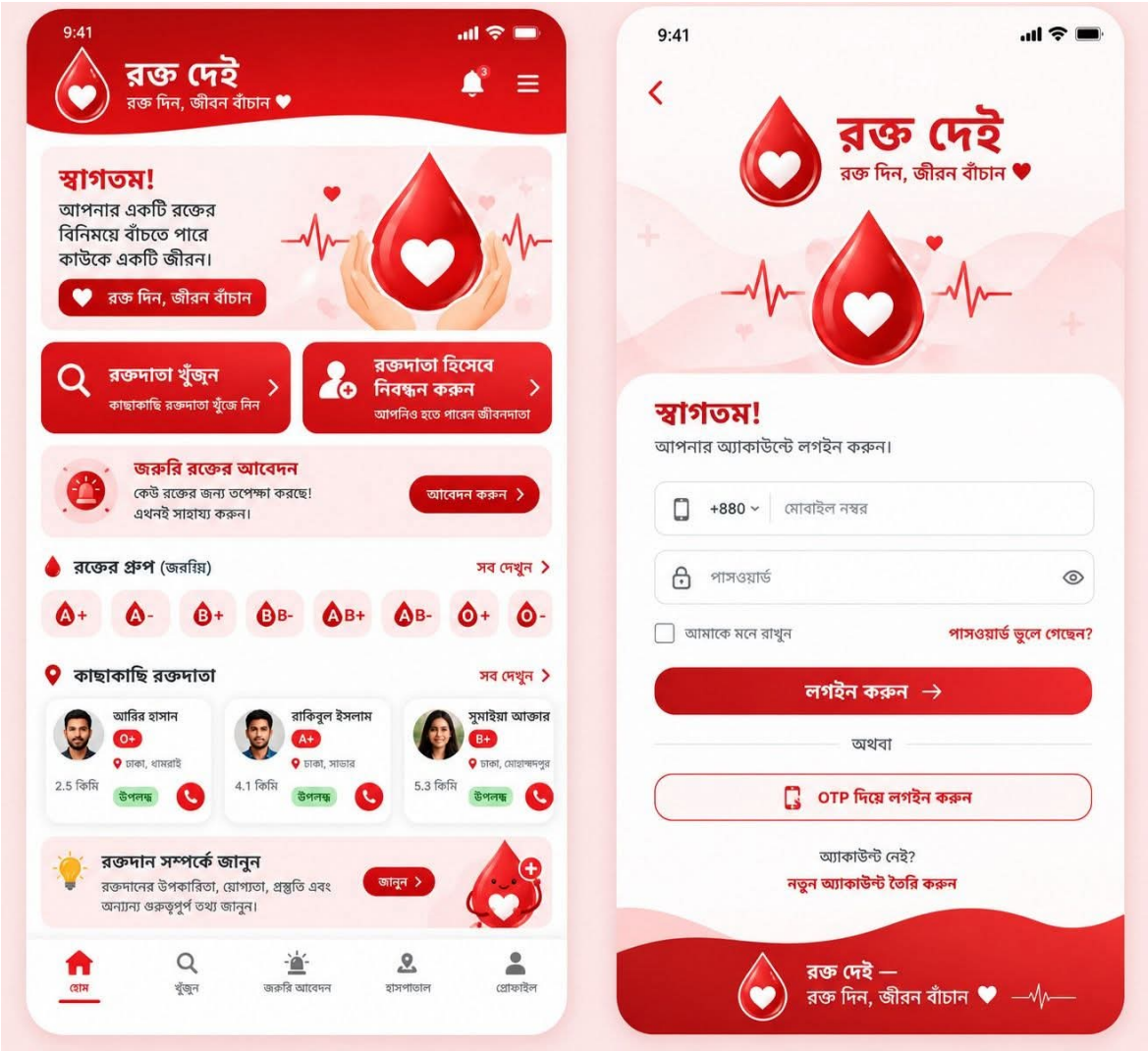


Figure 11.1 — Reference screenshot of a Bangla blood donor app (attached to the project brief). The red-on-white palette, phone-first login, and bottom navigation pattern are adapted for Jashore; the urgent-countdown and live-counter patterns are deliberately omitted.

Public Pages

Page	Purpose	Notes
Homepage	Primary entry point; four CTAs	Find Blood > Become a Donor > Request Blood > Find Hospital
Find Blood	Public donor search	No login required to search; login required to contact
Donor Registration	Become a donor	Requires account; asks for blood group, upazila, availability
Blood Request Submission	Submit a request	Requires account; duplicate-detection warning
How It Works	Plain-language explanation	Both donor and requester flows
Hospital Directory	List of verified hospitals	Public read; no PII exposed
About	Project background	Mission, team, governance, contact
Contact	Contact form	Routes to moderation queue

Page	Purpose	Notes
Privacy Policy	Required by law	Reviewed by counsel before launch
Terms of Service	Required by law	Reviewed by counsel before launch

Table 11.1 — Public pages. Every public page is server-rendered for fast first paint on slow networks; interactivity hydrates progressively.

Authenticated Pages

- **User Dashboard** — overview of the user's activity: open requests, recent responses, unread notifications, donor profile status (if applicable).
- **Donor Profile** — manage blood group, upazila, locality, availability, contact preferences, visibility, notification preferences. Reconfirm-availability button resets the availability timestamp.
- **My Blood Requests** — list of requests the user has submitted, with status, response count, and actions (cancel, view responses).
- **My Responses** — list of requests the user (as donor) has responded to, with response status and withdraw action.
- **Notification Settings** — per-channel opt-in (in-app, email, future SMS) and per-event-type granularity.
- **Account Settings** — phone, email, password (if applicable), account deletion workflow.

Administrative Pages

- **Admin Dashboard** — operational metrics, no fake counters.
- **Donor Management** — paginated donor list, search, suspend, reactivate. No bulk PII export.
- **Blood Request Management** — pending verifications, active requests, completed requests. Verify, reject, cancel actions audit-logged.
- **Facility Verification** — pending hospital verifications queue with evidence review and approve/reject workflow.
- **Abuse Reports** — queue of user-submitted reports with moderator actions (hide, suspend, dismiss).
- **User and Role Management** — user search, role assignment (guarded by the policy in Chapter 10).
- **Audit Logs** — filterable by actor, action, target, time range. Read-only; export to CSV for compliance review.
- **Reports** — operational reports: active donors by blood group, requests by upazila, response rate, time-to-arrangement.

Homepage CTA Priority

The homepage is the most important page on the platform. Its primary job is to route a stressed-out visitor to the right action in under five seconds. The four CTAs are presented in the order they are most likely needed: **Find Blood** first (a visitor looking for blood is in a hurry), **Become a Donor** second (a visitor willing to help), **Request Blood** third (a visitor who could not find a donor through search), and **Find a Hospital** fourth (a visitor who

needs to confirm where to donate). Each CTA is a large, high-contrast button with an icon and a one-line description; they are stacked vertically on mobile to remain thumb-reachable.

State Design

Every interactive surface must define five states: loading, empty, validation error, success, and server error. Loading states use skeleton screens rather than spinners (skeletons feel faster and do not imply that something is wrong). Empty states explain what the user would see if there were data and what action to take next. Validation errors are inline, per-field, and spoken by screen readers via aria-describedby. Success states confirm what happened and what to expect next. Server errors offer a retry button and a contact link; they never expose stack traces or technical details.

Performance Budget

The performance budget targets a Lighthouse mobile score of 90+ on a mid-range Android device on a 3G connection. Concretely: first contentful paint under 1.5 seconds on 3G, time-to-interactive under 3.5 seconds, total JavaScript payload under 150 KB gzipped for the homepage. React Server Components are used aggressively to keep the client bundle small. Images are served via Next.js Image with AVIF and WebP fallbacks. No third-party analytics script loads above the fold.

Internationalization Readiness

Bangla localization is a future feature, but the architecture must not prevent it. All user-facing strings are extracted into a messages file keyed by a stable identifier; the React component tree renders via a component that takes a message key and parameters. The default locale is en; a bn locale directory is created empty so that adding Bangla is a translation exercise, not a refactor. Date, number, and time formatting use Intl with the user's locale. CSS uses logical properties (margin-inline-start rather than margin-left) so that an RTL or vertical-text locale switch does not break the layout.

DELIVERABLE H · CHAPTER 12

Security and Privacy Assessment

This chapter is a threat-model review and a data-protection plan. It is written from the assumption that the platform will be attacked: by spammers submitting fake blood requests, by curious users trying to enumerate donor phone numbers, by malicious actors attempting to harvest donor data for non-medical contact, and by compromised accounts attempting to escalate privileges. The mitigations below are layered: no single control is assumed to hold, and a breach of any one control should be caught by the next.

Threat Model

Threat	Likelihood	Impact	Primary Mitigation
Credential stuffing / OTP abuse	High	High	Rate limit OTP + login; lockout after 5 failed attempts; anomaly detection
Broken access control (IDOR)	Medium	Critical	NestJS guards + RLS; ownership checks in policies; integration tests per route
Donor PII enumeration via search	High	High	No PII in search results; rate limit search; require login; honeypot donors
SQL injection	Low	Critical	Prisma parameterized queries only; no raw SQL in app code; CodeQL in CI
XSS in user-generated content	Medium	High	React default escaping; sanitize on render not on input; CSP headers
CSRF on state-changing endpoints	Medium	High	SameSite=Lax cookies; double-submit token for high-risk actions
Fake blood request spam	High	Medium	Daily submission limit per user; duplicate detection; moderator queue
Account enumeration via login	Medium	Medium	Generic error messages; uniform response timing; rate limit
Donor harassment via platform	Medium	High	No direct contact channel; donor opts in per request; abuse report workflow
Privileged role escalation	Low	Critical	Server-side role checks; appointment-only roles; audit log on every change
Insecure file upload (hospital docs)	Medium	High	Supabase Storage with private bucket; virus scan; size and type whitelist
Sensitive info in logs	Medium	High	Structured logging with redaction; no PII in logs; log review in CI
Database backup exposure	Low	Critical	Supabase-managed encrypted backups; no manual backups to disk; access review

Table 12.1 — Threat model. Likelihood and impact are qualitative ratings calibrated for a district-scoped pilot.

Recommended Controls

Input Validation

Every API request body is validated by a Zod schema shared with the frontend. Schemas enforce field types, length limits, enum membership, and cross-field invariants (e.g. "required_by must be in the future"). Validation failures return RFC 7807 problem-details responses. No user input is trusted until it has passed validation; this includes data that the frontend has already validated, because the frontend can be bypassed.

Secure Headers

The API returns a strict security header set: Content-Security-Policy restricting script and connect sources, X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: strict-origin-when-cross-origin, Permissions-Policy denying camera, microphone, and geolocation by default. The frontend uses a relaxed CSP

that allows Vercel-hosted scripts and the API origin only.

Rate Limiting

Per-endpoint rate limits are documented in Chapter 9 (Table 9.2). Limits are enforced via a sliding-window counter; during the pilot the counter is stored in PostgreSQL (simpler infrastructure), with a documented migration path to Redis when traffic justifies it. Abuse of a limit triggers a temporary block and an ABUSE_DETECTED audit log entry; repeated abuse triggers a moderator review.

Audit Logging

Every privileged action writes a row to the audit_logs table. The log is append-only: there is no UPDATE path, no DELETE path, and the table is not exposed via any public API endpoint. Administrators can query the log through a dedicated admin endpoint with pagination and filtering. The log captures actor, action, target type, target ID, IP address, user agent, and a JSON metadata field; **it never captures passwords, tokens, or credentials**.

Dependency Scanning

GitHub Dependabot is enabled on the repository and runs on every push. CodeQL analysis runs on every pull request. Critical advisories block merge to main. npm audit runs in CI; high-severity vulnerabilities fail the build. The dependency list is reviewed quarterly as part of the security review cadence.

Data Protection Plan

Data Minimization

The platform collects only the data needed to operate. Donor profiles capture blood group, location, and availability — not medical history, not medication, not travel history. Hospital directory entries capture publicly available information (name, address, phone) — not internal staffing data. Blood requests capture the minimum needed for a donor to decide whether to respond: blood group, units, hospital, general location, required time. Free-text fields are kept short and are reviewed for accidental PII leakage by the moderation team.

Consent

Consent is captured explicitly: a user becomes discoverable as a donor only after agreeing to the donor consent text; a user receives notifications only on channels they have opted into. Consent records are append-only: every grant and every withdrawal is a separate row, so the full consent history is reconstructable. A user may withdraw consent at any time; withdrawal takes effect immediately (donor profile removed from search, notifications stopped) even though the underlying data is retained for the audit window.

Retention, Correction, Deletion, Access

A user may request a copy of all data the platform holds about them (access request), may correct any inaccurate data through the account settings UI, and may delete their account through the deletion workflow. Deletion is a soft delete followed by a hard delete after 90 days. Audit logs are retained indefinitely (subject to legal review) because they are needed for abuse investigation. The full data-retention schedule is in Chapter 8, Section 8.5.

Bangladesh Legal and Regulatory Review

> **Not legal advice**
>
> The following is a preliminary identification of legal frameworks that may apply. It is not a compliance claim. A qualified Bangladesh-licensed lawyer must review this section before public launch. The platform must not claim compliance with any law until that review is complete.

Three legal instruments are likely to apply to a platform handling personal data in Bangladesh. **The Information and Communication Technology Act, 2006** (and its 2013 amendment) addresses computer crime, unauthorized access, and data damage; Section 54 in particular has been used in cases involving unauthorized disclosure of personal data. **The Information and Communication Technology (Implementation of Right to Information) Rules, 2010** may bear on what records the platform must produce on request. **The draft Personal Data Protection Act** (introduced 2022, still under review as of the date of this document) would, if enacted, impose consent, breach notification, and cross-border transfer requirements similar to the EU GDPR. The architecture in this document is designed to be forward-compatible with the draft Act's requirements (consent records, data minimization, deletion workflow, audit trail), but compliance cannot be claimed until the Act is final and counsel has reviewed the implementation.

DELIVERABLE I · CHAPTER 13

Testing Strategy

The testing strategy covers six test layers: unit, integration, end-to-end, security, cross-browser, and responsive. Each layer has a defined scope, a recommended tool, and a CI gate. Tests are written alongside the code they cover, not as a separate phase; pull requests that lower coverage below the agreed threshold (80% line coverage for the API, 60% for the web app) are blocked from merge.

Test Layers

Layer	Scope	Tool	CI Gate
Unit	Business logic, validation, authorization policies, helpers	Vitest (API), Vitest (web)	Must pass; coverage threshold
Integration	Supabase/Postgres operations, auth token verification, donor search, request lifecycle	Vitest + test database (Supabase branch)	Must pass on every PR
End-to-end	Registration, login, donor search, request submission, response, completion	Playwright	Must pass on every PR against preview deploys
Security	Unauthorized access, role escalation, rate-limit, data-exposure, request abuse	OWASP ZAP baseline scan + custom Playwright scripts	Weekly + on release
Cross-browser	Chrome, Firefox, Edge, Safari, Android Chrome	Playwright multi-browser + BrowserStack for Safari iOS	On release

Layer	Scope	Tool	CI Gate
Responsive	Small mobile (360px), tablet (768px), desktop (1280px)	Playwright viewport matrix	On release

Table 13.1 — Test layers, tools, and CI gates.

Accessibility Testing

Accessibility is a first-class concern because the platform serves people in distress who may have visual, motor, or cognitive impairments. Automated checks run in CI via axe-core on every Playwright page snapshot; manual keyboard-only navigation testing is performed before each release. Color contrast is verified at WCAG AA (4.5:1 for body text). Forms are labelled, error messages are announced by screen readers, focus order is logical, and no action requires a hover-only device.

Slow-Network Testing

Because the pilot audience is on 2G/3G networks, the test suite includes a "slow network" profile: Playwright runs with network throttling set to "Slow 3G" (400 ms RTT, 1.6 Mbps down, 750 Kbps up). The test asserts that the homepage reaches first contentful paint within 3 seconds and that no critical flow (login, donor search, request submission) times out within 30 seconds.

Test Data

Integration and E2E tests use a dedicated Supabase project (separate from the dev and prod projects) that is reset before each test run. Test data is clearly fictional: phone numbers use the +880 0NNNN test ranges, names use clearly fake values (e.g. "Test Donor 1", "Fictional Hospital A"), and an is_fictional flag is set on every record. The test database is never used for staging or demo purposes.

> **Honest reporting rule**
>
> Tests must not be claimed to have passed unless they have actually been executed. CI status badges reflect the most recent run, not an aspirational state. A failing test blocks merge; a flaky test is quarantined and fixed within one sprint, not disabled silently.

DELIVERABLE J · CHAPTER 14

Deployment Architecture and Free-Tier Cost Analysis

The deployment topology has four hosting surfaces (Vercel for the frontend, Render for the API, Supabase for the database and auth, GitHub for source control and CI) and three environments per surface (production, preview, local). This chapter documents the build and deploy pipeline for each, the environment-variable inventory, the free-tier limits, and the concrete numeric triggers that should prompt a move to a paid plan.

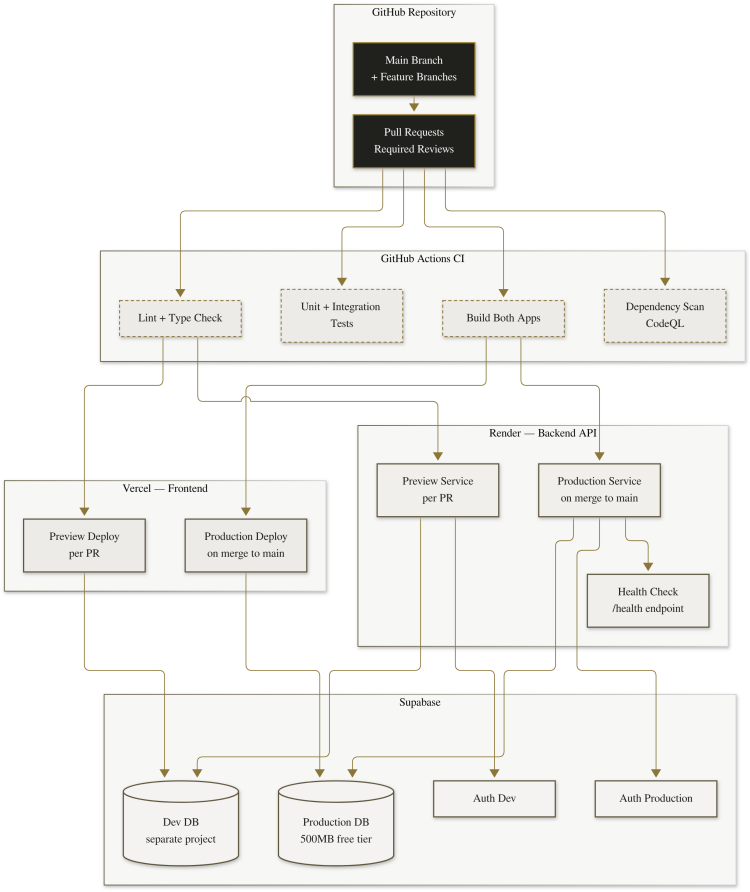


Figure 14.1 — Deployment architecture. Pull requests trigger preview deploys on both Vercel and Render; merges to main trigger production deploys. Production services point at the production Supabase project; preview services point at a separate dev project.

Frontend: Vercel

The web app is deployed on Vercel from the apps/web directory. The build command is pnpm build; the output directory is .next. Environment variables are configured in the Vercel dashboard and scoped per environment (Production, Preview, Development). Preview deployments are created automatically for every pull request; the preview URL is posted as a comment on the PR. Production deployments happen on merge to main.

Backend: Render

The API is deployed on Render as a Web Service from the apps/api directory. The build command is pnpm build; the start command is node dist/main.js. A health check endpoint (GET /health) returns 200 OK and is polled by Render every 30 seconds. The service is configured for graceful shutdown: on SIGTERM, the server stops accepting new connections, drains in-flight requests for up to 10 seconds, then exits. Render's free tier sleeps the service after 15 minutes of inactivity; the first request after sleep takes approximately 30 seconds to wake the service, which is unacceptable for an emergency blood platform — see the cost-trigger table below.

Database and Auth: Supabase

Supabase hosts the PostgreSQL database, the Auth service, and the Storage service (for hospital verification document uploads). Two Supabase projects are maintained: jbn-prod for production and jbn-dev for development and preview deploys. Each project has its own database, auth, and storage. Connection pooling is enabled via Supavisor; the NestJS API connects via the pooler URL on port 6543, not directly to Postgres on port 5432. Database migrations are applied via Prisma migrate from a CI job with the migration role key, not from the application runtime. Backups are Supabase-managed (daily snapshots with 7-day retention on the free tier); a quarterly restore test is performed into a sandbox project.

GitHub and CI

The repository is hosted on GitHub. The main branch is protected: pull requests require at least one approving review, CI must pass, and the branch must be up to date. The CI pipeline runs on every pull request and on every merge to main. It executes lint, typecheck, unit tests, integration tests, build, and a security scan (CodeQL + npm audit + OWASP ZAP baseline). Preview deploys are triggered after CI passes; production deploys are triggered on merge to main. Deploy approvals for production are manual during the pilot phase (a GitHub Environment with required reviewers).

Free-Tier Limits and Upgrade Triggers

Service	Free-tier limit	Upgrade trigger	Paid plan (approx)
Vercel (frontend)	100 GB bandwidth / month; 100 GB-hours	Bandwidth > 80 GB for two consecutive months	Pro ~$20/mo
Render (backend)	750 hours / month; sleeps after 15 min idle	Any production traffic (sleep is unacceptable for emergencies)	Starter ~$7/mo
Supabase (DB)	500 MB storage; pauses after 7 days inactivity	Storage > 400 MB OR > 50 active users	Pro ~$25/mo
Supabase (auth)	50,000 MAU; 4 SMS OTPs/sec	MAU > 40,000 OR SMS volume > 1,000/mo	Pro ~$25/mo (incl. SMS credits)
Supabase (storage)	1 GB storage; 2 GB egress / month	Storage > 800 MB OR egress > 1.5 GB	Pro ~$25/mo (bundled)

Service	Free-tier limit	Upgrade trigger	Paid plan (approx)
GitHub Actions	2,000 minutes / month (free)	Minutes > 1,800 for two consecutive months	Pro ~$4/user/mo

Table 14.1 — Free-tier limits and upgrade triggers. The Render sleep behaviour is the most urgent trigger: it forces a paid plan the moment the pilot accepts real emergency requests.

Environment Variable Inventory

The following environment variables are required. **Placeholders only — never real credentials in code or documentation.** The real values are configured in the Vercel, Render, and Supabase dashboards and are never committed.

```
# Frontend (apps/web, Vercel)
NEXT_PUBLIC_API_URL=https://api.jashore-blood-network.org
NEXT_PUBLIC_SUPABASE_URL=https://jbn-prod.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...            # anon key, safe to expose

# Backend (apps/api, Render)
DATABASE_URL=postgresql://...@db.jbn-prod.supabase.co:6543/postgres  # pooler
DIRECT_URL=postgresql://...@db.jbn-prod.supabase.co:5432/postgres    # migrations
SUPABASE_URL=https://jbn-prod.supabase.co
SUPABASE_SERVICE_ROLE_KEY=eyJ...               # NEVER in frontend
SUPABASE_JWT_SECRET=...                        # for token verification
JWT_COOKIE_DOMAIN=.jashore-blood-network.org
CORS_ALLOWED_ORIGINS=https://jashore-blood-network.org,https://*.vercel.app
EMAIL_FROM=noreply@jashore-blood-network.org
RESEND_API_KEY=re_...
RATE_LIMIT_REDIS_URL=...                       # optional during pilot
```

DELIVERABLE K · CHAPTER 15

Development Roadmap with Acceptance Criteria

The roadmap is organized into eleven phases (Phase 0 through Phase 11). Each phase has a defined objective, a set of dependencies on prior phases, a list of deliverables, the decisions that must be made before the phase can start, the acceptance criteria that must be met before the phase is considered complete, the testing requirements, and the identified risks. Effort estimates are in person-days and assume a single full-stack engineer; parallelism reduces wall-clock time proportionally.

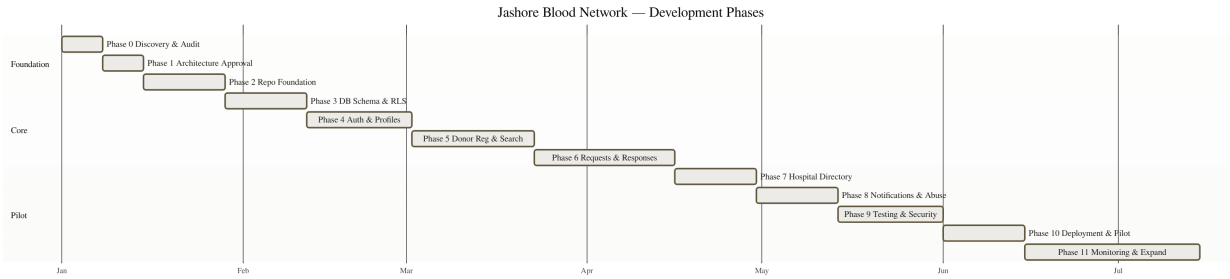


Figure 15.1 — Phase timeline (indicative). Phases 0 and 1 are covered by this document; Phases 2 through 11 follow approval.

Phase Summaries

Phase	Objective	Effort	Acceptance Criteria
0. Discovery & Audit	Inspect workspace, document findings	2 days	This document; workspace audit in Ch.2
1. Architecture Approval	Present plan, get explicit sign-off	3 days	Stakeholder approval recorded in repo
2. Repo Foundation	Monorepo skeleton, tooling, CI	5 days	pnpm install + pnpm build + pnpm test all pass on a fresh clone
3. DB Schema & RLS	Prisma schema, migrations, RLS policies	5 days	Migrations apply cleanly to fresh Supabase project; RLS tests pass
4. Auth & Profiles	Login, OTP, session, profile CRUD	6 days	E2E login flow works; profile CRUD unit + integration tests pass
5. Donor Reg & Search	Donor profile, search, filtering	7 days	Search returns no PII; performance budget met; E2E donor journey passes
6. Requests & Responses	Blood request lifecycle, donor response	8 days	Full lifecycle E2E passes; duplicate detection works; rate limits enforced
7. Hospital Directory	Hospital CRUD, verification workflow	5 days	Admin can verify a hospital; public directory shows verified only
8. Notifications & Abuse	In-app + email notifications, abuse reports	5 days	Notification queue drains; abuse report lifecycle works
9. Testing & Security	Full test suite, security review, a11y audit	6 days	Lighthouse 90+; OWASP ZAP clean; axe-core zero violations
10. Deployment & Pilot	Production deploy, controlled pilot	5 days	Pilot users complete end-to-end flow; no P0 issues for 7 days
11. Monitoring & Expand	Observability, on-call, expansion planning	10 days	SLO dashboards live; on-call rota agreed; expansion plan documented

Table 15.1 — Phase summaries. Effort is in person-days for a single full-stack engineer. Phases 0 and 1 are the current engagement; Phases 2 through 11 begin after approval.

Phase 0 and Phase 1 (Current Engagement)

Phase 0 (discovery and workspace audit) and Phase 1 (architecture approval) are the scope of this document. Phase 0 is complete: the workspace audit is in Chapter 2 and the findings are integrated into subsequent chapters. Phase 1 ends when stakeholders record explicit approval in the repository (e.g. an APPROVAL.md file signed by the designated approvers). Without that approval, Phase 2 does not start.

Phase 2: Repo Foundation (Recommended Next Task)

Phase 2 is the recommended next task after approval. Its scope is deliberately minimal: it produces a working monorepo skeleton with tooling and CI, but no business logic. Concretely, Phase 2 delivers: a pnpm workspace with apps/web (Next.js), apps/api (NestJS), and the three shared packages; Turborepo configuration; TypeScript strict mode in all packages; ESLint and Prettier configuration; Vitest setup with a sample passing test in each package; a GitHub Actions workflow that runs lint, typecheck, test, and build on every pull request; a .gitignore that covers .env, node_modules, .next, dist, and .turbo; a .env.example with placeholders; and a README with setup instructions. **No Supabase project is created in Phase 2.** No database schema, no auth integration, no UI — those belong to Phases 3 and 4.

Dependency-Aware Sequencing

The phase ordering is dependency-aware. Phase 3 (DB schema) requires Phase 2 (repo foundation) because the Prisma schema lives in the repo. Phase 4 (auth) requires Phase 3 because auth depends on the profiles table. Phase 5 (donor search) requires Phase 4 because donor visibility is gated by authentication. Phase 6 (requests and responses) requires Phase 5 because the request form references the donor directory. Phases 7 and 8 may run in parallel after Phase 6. Phase 9 (testing and security) requires Phases 4 through 8 to be substantially complete. Phase 10 (deployment) requires Phase 9. Phase 11 (monitoring) overlaps with Phase 10 and continues indefinitely.

> **MVP over completeness**
>
> The roadmap prioritizes a small, reliable MVP. Anything not in Phases 2 through 10 is deferred. The deferred items include SMS notifications, multi-district expansion, blood bank inventory, the Android app, the Bangla interface, and the volunteer coordination workflows. These are tracked as future features in Chapter 6.I and will be revisited after the pilot has been running for at least three months.

DELIVERABLE L · CHAPTER 16

Open Questions and Unresolved Decisions

The questions below are explicitly open. They are listed here so that stakeholders can answer them in one batch rather than discovering them piecemeal during implementation. Each question has a context line, the recommended default if no answer is given, and a decision deadline tied to a phase. Questions whose deadline passes without an answer are resolved using the recommended default and recorded in the decision log.

ID	Question	Recommended Default	Decision Deadline
Q-L1	Authentication strategy: phone+OTP only, email+password only, or hybrid?	Hybrid: phone primary, email secondary. Pilot may start phone-only.	Before Phase 4
Q-L2	Repository root: separate Git repo for the application, or sub-directory inside the existing workspace?	Separate GitHub repository (jashore-blood-network); current workspace stays for tooling.	Before Phase 2

ID	Question	Recommended Default	Decision Deadline
Q-L3	Distance-based donor search: PostGIS radius vs. upazila-only filter?	Upazila-only filter for MVP; PostGIS deferred until multi-district.	Before Phase 5
Q-L4	Image upload strategy for hospital verification documents: Supabase Storage vs. external (e.g. Cloudinary)?	Supabase Storage with a private bucket; AVIF/WebP via Next.js Image.	Before Phase 7
Q-L5	Bangla localization: which i18n library, and when to launch?	next-intl; launch Bangla after pilot is stable (~3 months post-launch).	Before Phase 11
Q-L6	Multi-district expansion: when, and what is the trigger?	Trigger: 3 months of stable operation with > 100 active donors. Architecture already supports it.	Phase 11
Q-L7	Pilot scope: how many donors and hospitals before public launch?	20 verified donors across at least 4 upazilas; 2 verified hospitals. Soft launch.	Before Phase 10
Q-L8	Governance: who owns the Super Administrator role?	A named individual + a named backup. Documented in the governance section of the README.	Before Phase 10
Q-L9	Funding model: NGO, government partnership, hybrid?	NGO-led with in-kind sponsorship from local hospitals. Funding plan documented before pilot.	Before Phase 10
Q-L10	Medical disclaimer wording: has it been reviewed by counsel?	Use the disclaimer in Chapter 1 as a starting point; counsel must approve before launch.	Before Phase 10
Q-L11	SMS provider for OTP and notifications: which BD-reachable provider, what budget?	Defer SMS to Phase 8+; pilot uses email + in-app only. Resolve provider before Phase 8.	Before Phase 8
Q-L12	Observability stack: Sentry + Logflare + Vercel Analytics, or a single provider?	Sentry for errors, Supabase logs for DB, Vercel Analytics for traffic. Consolidate later.	Before Phase 11

Table 16.1 — Open questions. Recommended defaults are applied if no decision is recorded by the deadline.

DELIVERABLE M · CHAPTER 17

Risk Register with Mitigations

The risk register below covers technical, security, operational, regulatory, social, and governance risks. Each risk has a likelihood (Low/Medium/High), an impact (Low/Medium/High/Critical), a mitigation, an owner role, and a trigger condition that indicates the risk is materializing. The register is a living document: it is reviewed at the end of every phase and updated with any new risks discovered during implementation.

Risk	L	I	Mitigation	Owner	Trigger
Free-tier Render sleeps, causing 30s cold starts	H	H	Move to Render Starter (~$7/mo) before pilot accepts real requests	DevOps	Any production request
Supabase DB pause after 7 days inactivity (free tier)	M	H	Move to Supabase Pro before pilot; or daily keepalive job	DevOps	DB pause detected
Donor PII leak via search or API	M	C	No PII in search results; RLS on all donor tables; integration tests per route	Security	Any PII found in logs or responses
Fake blood request spam	H	M	Rate limit; duplicate detection; moderator queue; user reputation	Moderator	> 10 fake requests / day
Role escalation via guard bypass	L	C	Server-side guards; RLS backstop; audit log on every privileged action	Security	Anomaly in audit log
No SMS provider reachable from BD within budget	M	M	Defer SMS; email + in-app only for pilot; resolve Q-L11 before Phase 8	Product	Pilot users request SMS
Low-end device performance issues	M	M	Performance budget; Lighthouse CI; RSC; image optimization	Frontend	Lighthouse mobile < 80
2G/3G network timeout on critical flows	M	H	Slow-network test profile; progressive hydration; minimal JS	Frontend	Any flow times out in slow-network test
Bangladesh data law non-compliance	M	H	Counsel review before launch; forward-compatible architecture	Legal	Counsel identifies gap
Donor harassment via platform	M	H	No direct contact channel; per-request opt-in; abuse report workflow	Moderator	First abuse report upheld
Volunteer coordinator burnout	M	M	Onboarding docs; rotation; recognition; manageable queue size	Community	Queue > 50 items / coordinator
Single-point-of-failure for Super Admin	M	C	Named backup admin; documented runbook; access review quarterly	Governance	Admin unavailable > 7 days
Database backup restore fails when needed	L	C	Quarterly restore test to sandbox; alert if restore time > 1h	DevOps	Restore test fails
Dependency vulnerability (zero-day)	M	H	Dependabot; CodeQL; npm audit; rapid patch process	Security	Critical advisory published
Donor availability data goes stale	H	M	30-day availability decay; reconfirm-availability UI; weekly sweep job	Product	> 30% donors stale

Risk	L	I	Mitigation	Owner	Trigger
Cross-district expansion too early	M	M	Multi-district deferred until Q-L6 trigger; architecture supports it	Product	Expansion request before pilot stable

Table 17.1 — Risk register. L = Likelihood, I = Impact (L/M/H/C). Risks are reviewed at every phase boundary.

Risk Review Cadence

The register is reviewed at the end of every phase. New risks discovered during implementation are added; risks that have been fully mitigated are marked as closed but retained for historical reference. The review is a 30-minute meeting attended by the lead engineer, the designated product owner, and (when security risks are on the agenda) the security owner. The output is a dated revision of this table committed to the repository.

DELIVERABLES N, O · CHAPTER 18

Proposed Foundation Files and Next Task

This chapter lists the exact files and folders that will be created during Phase 2 (Repo Foundation), once this plan is approved. It also specifies the recommended next task that can be approved independently: Phase 2 itself, scoped narrowly to monorepo skeleton + tooling + CI, with no business logic, no Supabase project, and no database schema. This narrow scope allows Phase 2 to start even if some open questions in Chapter 16 remain unresolved.

Phase 2 File Manifest

Path	Purpose
package.json	Root: devDependencies, scripts, packageManager field
pnpm-workspace.yaml	Workspace declaration: apps/*, packages/*
turbo.json	Task pipeline: build, test, lint, typecheck, dev
.gitignore	Covers .env, node_modules, .next, dist, .turbo, .DS_Store
.env.example	Placeholders only; never real credentials
.editorconfig	Editor consistency (indent, EOL, charset)
README.md	Project overview, setup, contribution guide link
CONTRIBUTING.md	Branch policy, PR template, code review expectations
apps/web/package.json	Next.js 16 + React 19 + TypeScript + Tailwind 4
apps/web/tsconfig.json	Strict mode, noUncheckedIndexedAccess, paths
apps/web/next.config.ts	App Router, image optimization, env validation
apps/web/tailwind.config.ts	Mobile-first breakpoints, brand palette tokens

Path	Purpose
apps/web/src/app/layout.tsx	Root layout, font loading, metadata
apps/web/src/app/page.tsx	Placeholder homepage (Phase 5 fills it in)
apps/api/package.json	NestJS 11 + TypeScript + Prisma + Zod
apps/api/tsconfig.json	Strict mode, decorators enabled, paths
apps/api/nest-cli.json	NestJS build configuration
apps/api/src/main.ts	Bootstrap, CORS, cookie parser, graceful shutdown
apps/api/src/app.module.ts	Root module; only ConfigModule in Phase 2
apps/api/src/config/env.validation.ts	Zod schema for env vars; fails fast on missing
packages/shared-types/package.json	Shared TypeScript types and enums
packages/shared-types/src/index.ts	Re-exports
packages/shared-types/src/enums.ts	BloodGroup, RequestStatus, ResponseStatus, Role
packages/validation/package.json	Shared Zod schemas
packages/validation/src/index.ts	Re-exports
packages/validation/src/donor.ts	Donor profile schema (placeholder)
packages/api-client/package.json	Typed fetch wrapper for web app
packages/api-client/src/index.ts	createClient(baseUrl) factory
database/.gitkeep	Placeholder; migrations land in Phase 3
database/seeds/.gitkeep	Placeholder; reference data lands in Phase 3
.github/workflows/ci.yml	Lint + typecheck + test + build on every PR
.github/PULL_REQUEST_TEMPLATE.md	Description, risk, testing checklist
docs/README.md	Index of the 12 docs (created in later phases)

Table 18.1 — Phase 2 file manifest. No business logic, no Supabase project, no database schema. Pure skeleton + tooling.

Phase 2 Acceptance Criteria

Phase 2 is considered complete when all of the following are true on a fresh clone of the repository:

1. pnpm install completes without errors on a machine with Node 20+ and pnpm 9+.
2. pnpm build builds all three packages and both apps without errors.
3. pnpm test runs a sample passing test in each package and exits zero.
4. pnpm lint and pnpm typecheck pass on all packages.
5. A pull request opened against main triggers the CI workflow, which runs all four tasks above and reports success.
6. A preview deployment of the web app is created automatically on Vercel and returns a 200 OK on /.

7. A preview deployment of the API is created automatically on Render and returns 200 OK on GET /health.
8. No environment variable is committed; .env.example is the only env file in the repository.

Recommended Next Task for Independent Approval

> **Approval gate**
>
> The recommended next task is **Phase 2 (Repo Foundation)**, scoped as described above. Approval of Phase 2 authorizes the creation of the files in Table 18.1, the initialization of a GitHub repository, and the configuration of Vercel and Render preview projects. It does **not** authorize creation of the production Supabase project, deployment to production, or implementation of any feature logic. Those await separate approval at the Phase 3 gate.

Phase 2 can be approved independently of the open questions in Chapter 16. None of the questions Q-L1 through Q-L12 block the creation of the monorepo skeleton. The decisions that do block subsequent phases (auth strategy, distance search, SMS provider) can be made in parallel with Phase 2 work, with their deadlines gating the corresponding later phases.

After Phase 2

On completion of Phase 2, the next approval gate is Phase 3 (DB Schema and RLS). Phase 3 requires the Supabase project to be created, the Prisma schema to be written, migrations to be applied to the dev project, and RLS policies to be in place. It is a substantial phase with security implications and should be reviewed carefully. The deliverable at the Phase 3 gate is a pull request containing the Prisma schema and migration files, reviewed by both an engineer and the security owner.

Document Status

This document is the complete Phase 0 and Phase 1 deliverable. It is submitted for stakeholder review. No further work is performed until approval is recorded. The next step is for the designated approvers to record their decision in the repository, after which Phase 2 may begin.